

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [ ]: # Your answer here
The problem requires writing a function that takes a list of integers and identifies the missing numbers in the sequence from 0 to the maximum number in the list. If all numbers are present, the goal is to efficiently find and return these missing numbers.
```

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

```
In [ ]: # Your answer here

#Using the "test case" section of my partner's code:
print(missing_num([4, 0, 2, 1, 6]))
# Expected output: [3, 5]
```

- Copy the solution your partner wrote.

```
In [ ]: # Your answer here

from typing import List

def missing_num(nums: List[int]) -> List[int]:
    n = max(nums) # Find the max number in the list
    num_set = set(nums) # Convert list to a set for O(1) lookups
    missing_numbers = [i for i in range(n + 1) if i not in num_set] # Find missing numbers
    return missing_numbers if missing_numbers else -1 # Return -1 if no numbers are missing

# Test cases
print(missing_num([0, 2])) # Output: [1]
print(missing_num([5, 0, 1])) # Output: [2, 3, 4]
print(missing_num([6, 8, 2, 3, 5, 7, 0, 1, 10])) # Output: [4, 9]
print(missing_num([3, 0, 1, 4, 6])) # Output: [2, 5]
print(missing_num([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])) # Output: []
```

- Explain why their solution works in your own words.

```
In [ ]: # Your answer here
The function first finds the maximum number in the list. Then, it converts the list to a set. It iterates through all numbers from 0 to the maximum number and checks which ones are missing. Finally, it returns a list of missing numbers or -1 if there are none. This approach is efficient because it uses a set for O(1) lookups.
```

- Explain the problem's time and space complexity in your own words.

```
In [ ]: # Your answer here
Time Complexity:
Finding the maximum number:  $O(n)$ 
Creating a set:  $O(n)$ 
Iterating from 0 to max(nums):  $O(n)$  in the worst case
Checking membership in a set:  $O(1)$  per lookup
Overall:  $O(n)$ 

Space Complexity:
Storing the set:  $O(n)$ 
Storing the missing numbers: Worst case  $O(n)$  if all numbers are missing
Overall:  $O(n)$ 
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [ ]: # Your answer here
The solution is efficient and correctly identifies missing numbers. However, one po
handling negative numbers or ensuring input validation. If negative numbers are inc
as the problem assumes non-negative integers. Additionally, an alternative approach
complete sequence to find missing numbers more efficiently in specific cases.
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
In [ ]: # Your answer here
Working on Assignment 1 was a great exercise in problem-solving and coding efficien
Initially, I struggled with understanding how to efficiently resolve the tree probl
I also learned how to evaluate code for efficiency and clarity. The biggest challen
Through this process, I realized the importance of writing clean, efficient,
Additionally, explaining the solution in my own words reinforced my understanding o
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable

- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔥 Please review our **Assignment Submission Guide** 🔥 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

